

Amazon Redshift — Cheat Sheet

Data warehouse cloud AWS, colonnaire et massivement parallèle (MPP) — l'essentiel pour un ingénieur. IST Workshop, juin 2026.

Ce que c'est & pourquoi c'est intéressant

- **OLAP, pas OLTP** : optimisé pour les agrégations/scans sur des millions-milliards de lignes, pas pour les transactions unitaires.
- **Stockage colonnaire compressé** : ne lit que les colonnes référencées → scans 10-100x plus rapides qu'un row-store.
- **MPP** : un leader node planifie, des compute nodes exécutent en parallèle ; storage découplé sur S3 (RA3/Serverless).
- **SQL ~PostgreSQL** : drivers JDBC/ODBC, psql, outils BI standards.
- **Intégration AWS** : S3 (COPY/UNLOAD/Spectrum), IAM, Glue, QuickSight, zero-ETL depuis Aurora.

Deux modes de déploiement

	Serverless	Provisionné (RA3)
Facturation	\$/RPU-heure, à la seconde (min. 60 s), 0 si idle	\$/nœud/heure, 24/7 (ou reserved)
Admin	quasi nulle	resize, WLM, maintenance
Idéal pour	charge intermittente, démarrage	charge constante, coût prévisible

Essai gratuit : \$300 de crédits / 90 jours pour les nouveaux comptes Redshift Serverless.

Démarrer en 5 étapes (Serverless)

1. Console AWS → Redshift → **Serverless** → créer namespace + workgroup (base capacity : 8 RPU min.).
2. Attacher au namespace un rôle IAM avec accès S3 (lecture pour COPY).
3. Ouvrir **Query Editor v2** (aucun client à installer).
4. Créer les tables, charger via `COPY` depuis S3.
5. Configurer des **usage limits** (RPU-heures max/mois) — jour 1, pas après la première facture.

Ordres de grandeur de coût

Poste	Prix indicatif (eu-central-1)
Compute Serverless	≈ \$0.39 / RPU-heure
Storage géré (RMS)	≈ \$0.024 / GB-mois
Exemple PME : 8 RPU × 3 h/j ouvré + 600 GB	≈ \$230 / mois

Pièges de facturation : minimum 60 s par réveil de l'endpoint ; pas de plafond par défaut ; le result cache est gratuit mais un `SELECT` mal écrit sur une grosse table ne l'est pas.

Design de tables (provisionné surtout)

```
CREATE TABLE orders (  
  order_id BIGINT,  
  order_date DATE,  
  amount DECIMAL(10,2)  
)  
DISTSTYLE AUTO -- ou KEY/EVEN/ALL  
DISTKEY (order_id) -- colocalise les joins  
SORTKEY (order_date); -- accélère les filtres par plage
```

- **DISTKEY** : colonne de jointure fréquente → évite la redistribution réseau.
- **SORTKEY** : colonne de filtre fréquente (souvent la date) → zone maps, scans réduits.
- En Serverless / `DISTSTYLE AUTO`, Redshift gère seul — vérifier avec `SVV_TABLE_INFO`.

Charger & exporter (le cœur du job)

```
-- Chargement massif parallèle depuis S3 (LA bonne méthode)  
COPY orders FROM 's3://bucket/data/'  
IAM_ROLE 'arn:aws:iam::123:role/RedshiftRole'  
FORMAT PARQUET; -- ou CSV GZIP, JSON 'auto'  
  
-- Export (porte de sortie anti lock-in)  
UNLOAD ('SELECT * FROM orders')  
TO 's3://bucket/export/orders_'  
IAM_ROLE '...' FORMAT PARQUET PARALLEL ON;
```

Anti-pattern : les `INSERT` ligne à ligne sont ~1000x plus lents que `COPY`. Toujours passer par S3 + `COPY` pour l'ingestion.

Requêter — spécificités utiles

```
-- Fenêtres + QUALIFY (filtre direct sur window fn)
SELECT canton, category, SUM(amount) AS ca,
       RANK() OVER (PARTITION BY canton
                   ORDER BY SUM(amount) DESC) AS rg
FROM orders GROUP BY 1,2 QUALIFY rg <= 3;

-- Requêter S3 sans charger (Spectrum / external
schema)
CREATE EXTERNAL SCHEMA ext
FROM DATA CATALOG DATABASE 'lake'
IAM_ROLE '...';
SELECT * FROM ext.raw_events LIMIT 10;

-- Désactiver le cache (pour benchmarker honnêtement)
SET enable_result_cache_for_session TO off;
```

Maintenance & diagnostic

```
VACUUM orders;           -- re-trie, récupère l'espace
ANALYZE orders;          -- met à jour les stats du
planner
EXPLAIN SELECT ...;      -- plan d'exécution

-- Tables système incontournables
SELECT * FROM SVV_TABLE_INFO;           -- taille, skew,
stats
SELECT * FROM SYS_QUERY_HISTORY         -- requêtes
récentes
ORDER BY start_time DESC LIMIT 20;
SELECT * FROM SVL_QLOG;                 -- historique
(provisionné)
```

Serverless lance VACUUM/ANALYZE automatiquement ; en provisionné, planifiez-les après les gros chargements.

Quand l'utiliser / l'éviter

✓ Utiliser	× Éviter
Analytique > 100 GB, agrégations lourdes	< quelques dizaines de GB (Postgres suffit)
BI / batch intermittent (→ Serverless)	OLTP, latence < 100 ms côté application
Stack déjà AWS (S3, IAM, Aurora)	Stratégie multi-cloud stricte
Équipe SQL sans DBA dédié	Budget non surveillé (coût variable)

Lock-in : l'évaluation honnête

- **Données** : faible — `UNLOAD ... FORMAT PARQUET` sort tout en format ouvert (frais egress AWS à part).
- **SQL** : moyen — dialecte ~Postgres mais COPY, DIST/SORTKEY, Spectrum et tables système sont propriétaires.
- **Pipelines & IAM** : fort — c'est le vrai coût de sortie.
- **Mitigation** : garder la source de vérité en Parquet sur S3 (lakehouse) ; Redshift reste un moteur remplaçable.

Checklist de mise en prod

- ☐ Usage limits + alarmes de coût (CloudWatch/Budgets)
- ☐ Rôle IAM minimal (pas de S3 full access)
- ☐ Ingestion par COPY batch (jamais d'INSERT unitaires)
- ☐ Snapshots/backup vérifiés (automatiques, mais tester la restauration)
- ☐ Données brutes archivées en Parquet sur S3

Références

- Docs & pricing : [aws.amazon.com/redshift · billing](https://aws.amazon.com/redshift/billing)
Serverless : docs.aws.amazon.com/redshift
- Architecture : « Amazon Redshift Re-invented », SIGMOD'22
- Tutoriels : AWS Big Data Blog, catégorie Redshift

IST Workshop — Groupe Redshift (V. Giordani et al.) — prix vérifiés juin 2026, à reconfirmer sur la page pricing.